# Create a VS Code Plugin to Auto-Generate C/C++ Unit Tests with AI

This guide shows a working MVP for a VS Code extension that reads selected C/C++ code or the current file, sends it to an AI API, and writes generated unit tests into a tests folder.

## 1. What the plugin does

```
VS Code Extension
■■■ Command: Generate C/C++ Unit Tests with AI
■■■ Reads selected code or current file
■■■ Sends code to AI API
■■■ Receives generated test code
■■■ Writes tests/<source_file>_test.cpp
```

Recommended default for a mixed C/C++ project: use GoogleTest. For pure C projects, Unity/CMock can be added later.

## 2. Install tools

```
npm install -g yo generator-code vsce
```

Create the extension project:

```
yo code
```

Choose:

```
New Extension (TypeScript)
```

Use this extension name:

```
cpp-unit-test-ai-generator
```

Then install dependencies:

```
cd cpp-unit-test-ai-generator
npm install
npm install openai
```

## 3. Project structure

```
cpp-unit-test-ai-generator/
■■■ package.json
■■■ tsconfig.json
■■■ src/
■  ■■■ extension.ts
■  ■■■ aiClient.ts
■  ■■■ promptBuilder.ts
■  ■■■ testWriter.ts
■  ■■■ fileUtils.ts
■■■ README.md
```

## 4. package.json

Replace package.json with this MVP version. It registers commands, settings, and the extension entry point.

```
{
"name": "cpp-unit-test-ai-generator",
"displayName": "C/C++ Unit Test AI Generator",
"description": "Generate C/C++ unit test cases using AI.",
"version": "0.0.1",
"publisher": "kevin",
"engines": {
"vscode": "^1.90.0"
```

```
},
"categories": [
"Other"
],
"activationEvents": [
"onCommand:cppUnitTestAi.generateTests",
"onCommand:cppUnitTestAi.setApiKey",
"onCommand:cppUnitTestAi.clearApiKey"
],
"main": "./out/extension.js",
"contributes": {
"commands": [
{
"command": "cppUnitTestAi.generateTests",
"title": "C/C++ Unit Test AI: Generate Tests"
},
{
"command": "cppUnitTestAi.setApiKey",
"title": "C/C++ Unit Test AI: Set API Key"
},
{
"command": "cppUnitTestAi.clearApiKey",
"title": "C/C++ Unit Test AI: Clear API Key"
}
],
"configuration": {
"title": "C/C++ Unit Test AI Generator",
"properties": {
"cppUnitTestAi.model": {
"type": "string",
"default": "gpt-5.1",
"description": "AI model used to generate unit tests."
},
"cppUnitTestAi.framework": {
"type": "string",
"default": "gtest",
"enum": [
"gtest",
"unity",
"cpputest"
],
"description": "Unit test framework to generate."
},
"cppUnitTestAi.testFolder": {
"type": "string",
"default": "tests",
"description": "Folder where generated test files are saved."
},
"cppUnitTestAi.overwriteExistingFile": {
"type": "boolean",
"default": false,
"description": "Overwrite existing test file if it already exists."
}
}
}
},
"scripts": {
"vscode:prepublish": "npm run compile",
"compile": "tsc -p ./",
"watch": "tsc -watch -p ./",
"package": "vsce package"
},
"devDependencies": {
"@types/node": "^20.0.0",
"@types/vscode": "^1.90.0",
"typescript": "^5.0.0",
"vsce": "^2.15.0"
},
"dependencies": {
"openai": "^6.0.0"
}
}
```

## 5. tsconfig.json

```
{
"compilerOptions": {
"module": "Node16",
"target": "ES2022",
"outDir": "out",
"lib": [
"ES2022"
],
"sourceMap": true,
"rootDir": "src",
"strict": true,
"moduleResolution": "Node16",
"esModuleInterop": true
},
"exclude": [
"node_modules",
".vscode-test"
]
}
```

## 6. src/fileUtils.ts

```
import * as path from "path";

export type SourceLanguage = "C" | "C++";

export function isSupportedCOrCppFile(fileName: string): boolean {
const ext = path.extname(fileName).toLowerCase();

return [
".c",
".h",
".cpp",
".cc",
".cxx",
".hpp",
".hxx"
].includes(ext);
}

export function detectSourceLanguage(fileName: string): SourceLanguage {
const ext = path.extname(fileName).toLowerCase();

if (ext === ".c") {
return "C";
}

if (ext === ".h") {
return "C";
}

return "C++";
}

export function buildTestFileName(
sourceFileName: string,
framework: string
): string {
const ext = path.extname(sourceFileName);
const baseName = path.basename(sourceFileName, ext);

if (framework === "unity") {
return `test_${baseName}.c`;
}

return `${baseName}_test.cpp`;
}
```

## 7. src/promptBuilder.ts

This file builds the AI prompt. The prompt tells the model to return only source code and to handle C/C++ details correctly.

```
import { SourceLanguage } from "./fileUtils";

export interface UnitTestPromptRequest {
framework: string;
fileName: string;
sourceLanguage: SourceLanguage;
sourceCode: string;
}

export function buildUnitTestPrompt(request: UnitTestPromptRequest): string {
return `
You are a senior C/C++ unit test engineer.

Generate unit tests for the provided ${request.sourceLanguage} code.

Return rules:
- Return only source code.
- Do not include markdown.
- Do not include explanation.
- Do not wrap the answer in triple backticks.

Framework:
- Use this framework: ${request.framework}

General requirements:
- Generate complete compilable unit test code.
- Include required headers.
- Use meaningful test names.
- Include normal test cases.
- Include edge cases.
- Include boundary cases.
- Include negative/error cases where possible.
- If expected value is unknown, add TODO comments.
- If external dependency exists, add TODO comments for mock/stub.
- If function has pointer arguments, create safe dummy values.
- If function has array arguments, create small test arrays.
- If function has struct arguments, create initialized struct values.
- If function has global state, include setup/teardown TODO comments.

C/C++ specific rules:
- Source file name: ${request.fileName}
- Source language: ${request.sourceLanguage}
- If source language is C and framework is gtest, create a .cpp test file and wrap C headers
with extern "C".
- If source language is C++ and class methods exist, use TEST_F when setup is useful.
- If source language is C, do not invent C++ production APIs.
- If framework is unity, generate C test code using TEST_ASSERT_* macros.
- If framework is cpputest, generate TEST_GROUP and TEST cases.

Important:
- Do not change the production code.
- Do not assume private functions are directly accessible unless they are visible in the
provided code.
- Prefer testing public functions declared in headers.
- Add TODO comments when mocks are required.

Input code:
${request.sourceCode}
`;
}
```

## 8. src/aiClient.ts

```
import OpenAI from "openai";
import { buildUnitTestPrompt } from "./promptBuilder";
import { SourceLanguage } from "./fileUtils";

export interface AiGenerateRequest {
```

```
apiKey: string;
model: string;
framework: string;
fileName: string;
sourceLanguage: SourceLanguage;
sourceCode: string;
}

export async function generateUnitTestsWithAI(
request: AiGenerateRequest
): Promise<string> {
const client = new OpenAI({
apiKey: request.apiKey
});

const prompt = buildUnitTestPrompt({
framework: request.framework,
fileName: request.fileName,
sourceLanguage: request.sourceLanguage,
sourceCode: request.sourceCode
});

const response = await client.responses.create({
model: request.model,
input: prompt
});

const text = response.output_text;

if (!text || text.trim().length === 0) {
throw new Error("AI returned empty response.");
}

return cleanupAiCode(text);
}

function cleanupAiCode(text: string): string {
let cleaned = text.trim();

cleaned = cleaned.replace(/^```[a-zA-Z0-9]*\s*/g, "");
cleaned = cleaned.replace(/```$/g, "");

return cleaned.trim() + "\n";
}
```

# 9. src/testWriter.ts

```
import * as vscode from "vscode";
import { buildTestFileName } from "./fileUtils";

export interface WriteTestFileRequest {
sourceDocument: vscode.TextDocument;
generatedCode: string;
framework: string;
testFolder: string;
overwriteExistingFile: boolean;
}

export async function writeGeneratedTestFile(
request: WriteTestFileRequest
): Promise<void> {
const workspaceFolder = vscode.workspace.getWorkspaceFolder(
request.sourceDocument.uri
);

const sourceFile = request.sourceDocument.fileName;
const testFileName = buildTestFileName(sourceFile, request.framework);

if (!workspaceFolder) {
const newDoc = await vscode.workspace.openTextDocument({
content: request.generatedCode,
language: request.framework === "unity" ? "c" : "cpp"
```

```
});

await vscode.window.showTextDocument(newDoc);
return;
}

const outputDir = vscode.Uri.joinPath(
workspaceFolder.uri,
request.testFolder
);

const outputFile = vscode.Uri.joinPath(outputDir, testFileName);

if (!request.overwriteExistingFile) {
const exists = await fileExists(outputFile);

if (exists) {
const answer = await vscode.window.showWarningMessage(
`${testFileName} already exists. Overwrite it?`,
"Overwrite",
"Open Existing",
"Cancel"
);

if (answer === "Open Existing") {
const doc = await vscode.workspace.openTextDocument(outputFile);
await vscode.window.showTextDocument(doc);
return;
}

if (answer !== "Overwrite") {
return;
}
}
}

await vscode.workspace.fs.createDirectory(outputDir);

await vscode.workspace.fs.writeFile(
outputFile,
Buffer.from(request.generatedCode, "utf8")
);

const newDoc = await vscode.workspace.openTextDocument(outputFile);
await vscode.window.showTextDocument(newDoc);

vscode.window.showInformationMessage(`Generated ${testFileName}`);
}

async function fileExists(uri: vscode.Uri): Promise<boolean> {
try {
await vscode.workspace.fs.stat(uri);
return true;
} catch {
return false;
}
}
```

# 10. src/extension.ts

```
import * as vscode from "vscode";
import * as path from "path";
import { generateUnitTestsWithAI } from "./aiClient";
import {
detectSourceLanguage,
isSupportedCOrCppFile
} from "./fileUtils";
import { writeGeneratedTestFile } from "./testWriter";

const SECRET_KEY = "cppUnitTestAi.openaiApiKey";

export function activate(context: vscode.ExtensionContext) {
```

```
const setApiKeyCommand = vscode.commands.registerCommand(
"cppUnitTestAi.setApiKey",
async () => {
await setApiKey(context);
}
);

const clearApiKeyCommand = vscode.commands.registerCommand(
"cppUnitTestAi.clearApiKey",
async () => {
await context.secrets.delete(SECRET_KEY);
vscode.window.showInformationMessage("API key cleared.");
}
);

const generateTestsCommand = vscode.commands.registerCommand(
"cppUnitTestAi.generateTests",
async () => {
await generateTests(context);
}
);

context.subscriptions.push(
setApiKeyCommand,
clearApiKeyCommand,
generateTestsCommand
);
}

export function deactivate() {}

async function setApiKey(
context: vscode.ExtensionContext
): Promise<void> {
const apiKey = await vscode.window.showInputBox({
prompt: "Enter your OpenAI API key",
password: true,
ignoreFocusOut: true
});

if (!apiKey || apiKey.trim().length === 0) {
vscode.window.showWarningMessage("API key was not saved.");
return;
}

await context.secrets.store(SECRET_KEY, apiKey.trim());
vscode.window.showInformationMessage("API key saved securely.");
}

async function generateTests(
context: vscode.ExtensionContext
): Promise<void> {
const editor = vscode.window.activeTextEditor;

if (!editor) {
vscode.window.showErrorMessage("No active editor found.");
return;
}

const document = editor.document;
const fileName = document.fileName;

if (!isSupportedCOrCppFile(fileName)) {
vscode.window.showErrorMessage(
"Please open a C/C++ source or header file."
);
return;
}

const apiKey = await context.secrets.get(SECRET_KEY);

if (!apiKey) {
const answer = await vscode.window.showErrorMessage(
```

```
"API key is missing. Please set it first.",
"Set API Key"
);

if (answer === "Set API Key") {
await setApiKey(context);
}

return;
}

const selectedText = document.getText(editor.selection);

const sourceCode =
selectedText.trim().length > 0
? selectedText
: document.getText();

if (!sourceCode.trim()) {
vscode.window.showErrorMessage("No source code found.");
return;
}

const config = vscode.workspace.getConfiguration("cppUnitTestAi");

const model = config.get<string>("model", "gpt-5.1");
const framework = config.get<string>("framework", "gtest");
const testFolder = config.get<string>("testFolder", "tests");
const overwriteExistingFile = config.get<boolean>(
"overwriteExistingFile",
false
);

const sourceLanguage = detectSourceLanguage(fileName);

await vscode.window.withProgress(
{
location: vscode.ProgressLocation.Notification,
title: "Generating C/C++ unit tests with AI...",
cancellable: false
},
async () => {
const generatedCode = await generateUnitTestsWithAI({
apiKey,
model,
framework,
fileName: path.basename(fileName),
sourceLanguage,
sourceCode
});

await writeGeneratedTestFile({
sourceDocument: document,
generatedCode,
framework,
testFolder,
overwriteExistingFile
});
}
);
}
```

## 11. Compile and run

```
npm run compile
```

If you get an OpenAI SDK issue, update the package:

```
npm install openai@latest
npm run compile
```

Run the extension locally: press F5 in VS Code. This opens an Extension Development Host window. Then open a C or C++ file and run the commands from the Command Palette.

```
Ctrl + Shift + P
C/C++ Unit Test AI: Set API Key
C/C++ Unit Test AI: Generate Tests
```

# 12. Example C++ input and output

## Input: calculator.cpp

```
#include "calculator.h"

int add(int a, int b) {
return a + b;
}

bool isPositive(int value) {
return value > 0;
}
```

## Generated GoogleTest output

```
#include <gtest/gtest.h>
#include "calculator.h"

TEST(CalculatorTest, AddPositiveNumbers) {
EXPECT_EQ(5, add(2, 3));
}

TEST(CalculatorTest, AddNegativeNumbers) {
EXPECT_EQ(-5, add(-2, -3));
}

TEST(CalculatorTest, AddWithZero) {
EXPECT_EQ(3, add(0, 3));
}

TEST(CalculatorTest, IsPositiveReturnsTrueForPositiveNumber) {
EXPECT_TRUE(isPositive(10));
}

TEST(CalculatorTest, IsPositiveReturnsFalseForZero) {
EXPECT_FALSE(isPositive(0));
}

TEST(CalculatorTest, IsPositiveReturnsFalseForNegativeNumber) {
EXPECT_FALSE(isPositive(-1));
}
```

# 13. Example C input and output

## Input: math_utils.c

```
#include "math_utils.h"

int add(int a, int b) {
return a + b;
}
```

## Generated GoogleTest output

```
#include <gtest/gtest.h>

extern "C" {
#include "math_utils.h"
}

TEST(MathUtilsTest, AddPositiveNumbers) {
```

```
EXPECT_EQ(5, add(2, 3));
}

TEST(MathUtilsTest, AddNegativeNumbers) {
EXPECT_EQ(-5, add(-2, -3));
}

TEST(MathUtilsTest, AddWithZero) {
EXPECT_EQ(3, add(0, 3));
}
```

For C code tested by GoogleTest, extern "C" prevents C++ name-mangling issues.

## 14. Add right-click menu option

Add this under contributes in package.json if you want to right-click in the editor and generate tests.

```
"menus": {
"editor/context": [
{
"command": "cppUnitTestAi.generateTests",
"when": "resourceExtname == .c || resourceExtname == .cpp || resourceExtname == .h ||
resourceExtname == .hpp",
"group": "navigation"
}
]
}
```

## 15. Package as .vsix

```
npm run package
```

Install locally:

```
code --install-extension cpp-unit-test-ai-generator-0.0.1.vsix
```

## 16. Recommended VS Code settings

```
{
"cppUnitTestAi.model": "gpt-5.1",
"cppUnitTestAi.framework": "gtest",
"cppUnitTestAi.testFolder": "tests",
"cppUnitTestAi.overwriteExistingFile": false
}
```

## 17. Company-code warning

If your C/C++ project contains company or confidential code, do not send it to a public AI API unless your company approves it. For production use, make the AI endpoint configurable so you can later point the extension to OpenAI, Azure OpenAI, an internal company AI service, or a local LLM.